



DRISHTI

ENTERPRISE TECHNICAL SPECIFICATION

Drishti — Technical Architecture Specification

Defensive Cybersecurity Platform — Complete Architecture & Reference Manual



Graph-Powered Attack Paths



Live Watch Agent



Dollar Risk Quantifier



AI Remediation Guardrails



URL Trust Scoring

PLATFORM VERSION

v0.2 Enterprise Edition

RELEASE DATE

August 2026

ARCHITECTURE TIER

Full-Stack Multi-Tenant



RESTRICTED /// INTERNAL ARCHITECTURE & TECHNICAL SPECIFICATION /// FOR AUTHORIZED USE ONLY

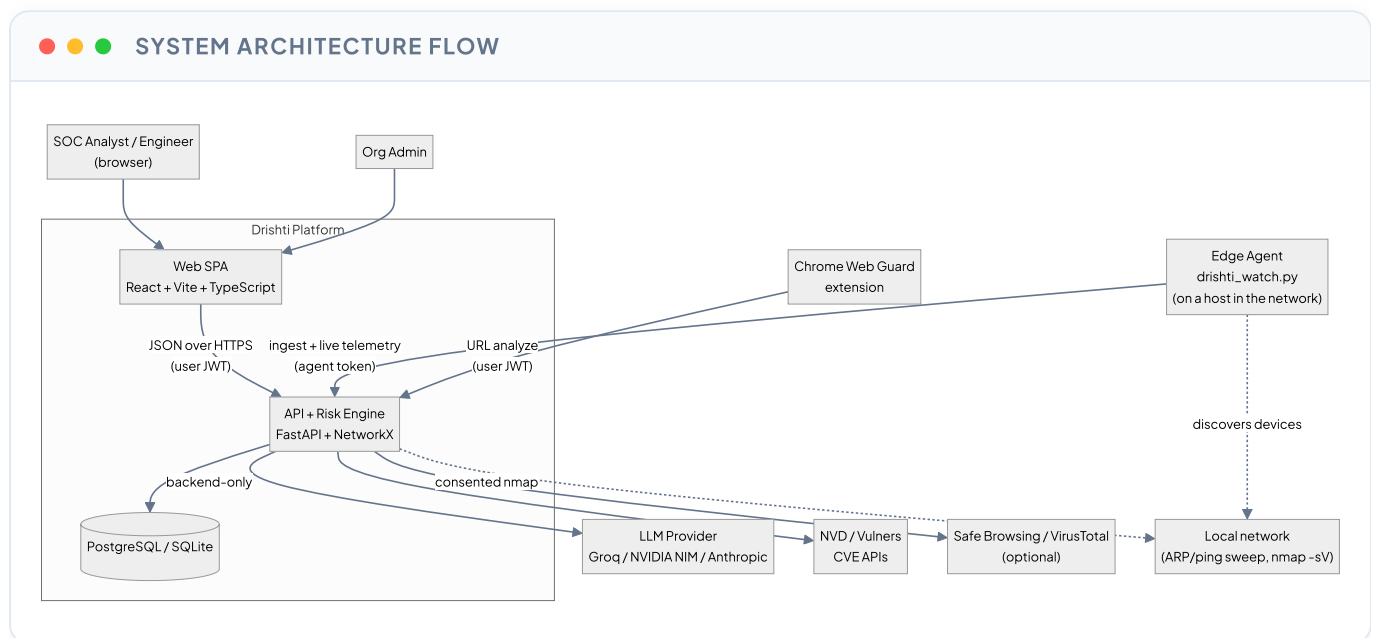
Drishti — Architecture

Reverse-engineered from the implemented product. C4-style views (context → container → component), data flow, deployment, and the design decisions that shaped them. Diagrams are Mermaid — they render on GitHub and in VS Code.

Last updated: 2026-08-17 — verified against source code.

1. System context (C4 L1)

Who and what talks to Drishti.

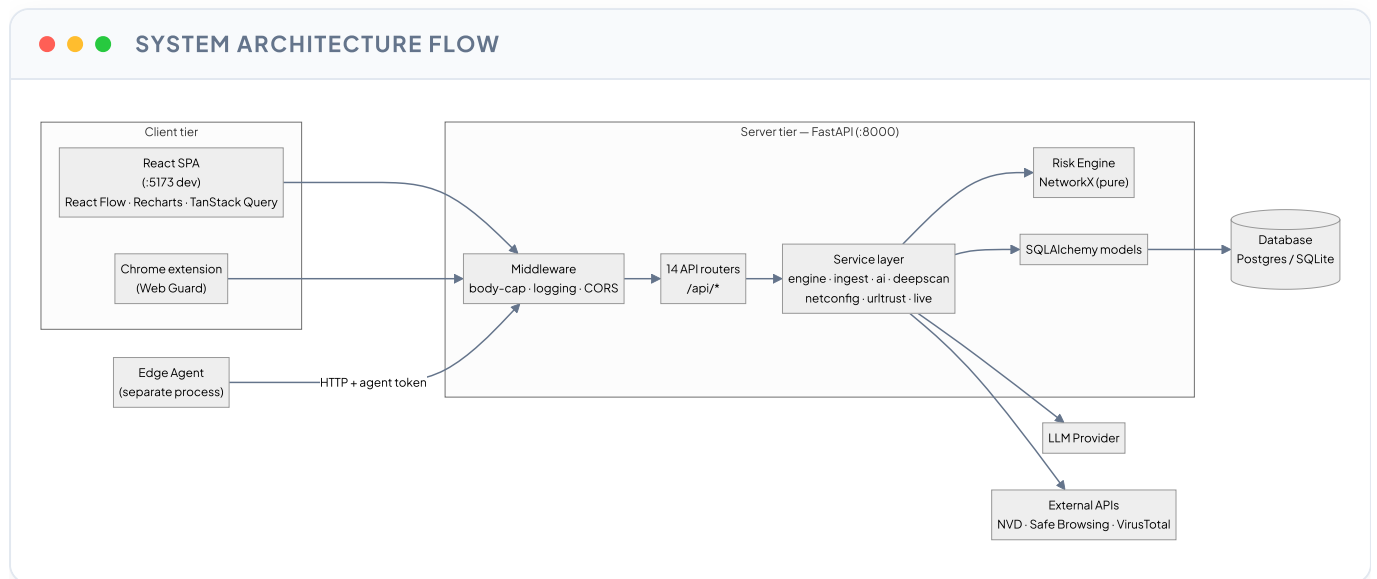


Trust boundaries: the browser and extension carry a **user JWT**; the edge agent carries a hashed **agent token**; the LLM key lives **only** on the backend. Every scan of the LAN is either the agent's consented device sweep or a consent-gated, RFC1918-only deep scan.

External integrations (all outbound from backend, key-isolated):

- **LLM:** Groq (default), NVIDIA NIM (`meta/llama-3.3-70b-instruct`), or Anthropic (`claude-sonnet-5`). Configured via `AI_PROVIDER` + `AI_MODEL` env vars. Structured JSON output via provider-specific schema enforcement.
- **CVE lookup:** NVD free REST API (no key needed, rate-limited ~5 req/30s) or Vulners (if `VULNERS_KEY` set).
- **URL reputation:** Google Safe Browsing + VirusTotal (both optional; absent = `configured:false`).

2. Container view (C4 L2)

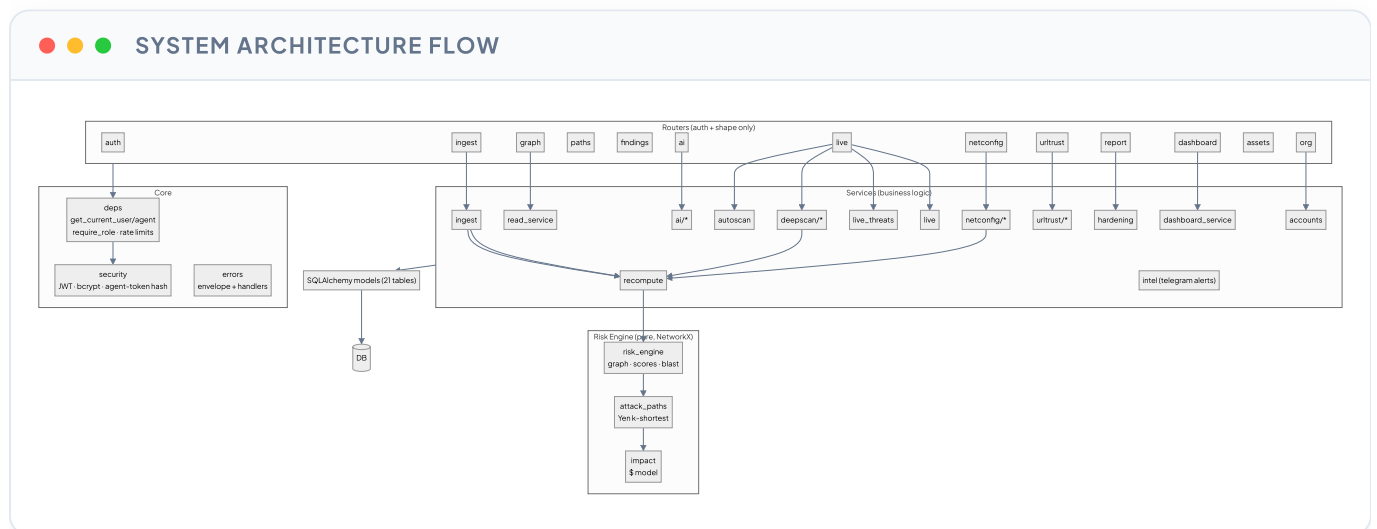


Key files:

- `server/app/main.py` — app assembly, lifespan bootstrap, middleware stack
- `server/app/config.py` — env settings (fail-closed JWT secret)
- `server/app/db.py` — engine/session + get_db dependency
- `server/app/db_init.py` — create_all + reconcile_columns
- `server/app/core/errors.py` — error envelope + handler registration

3. Backend component view (C4 L3)

Router → service → model, plus the pure engine core.

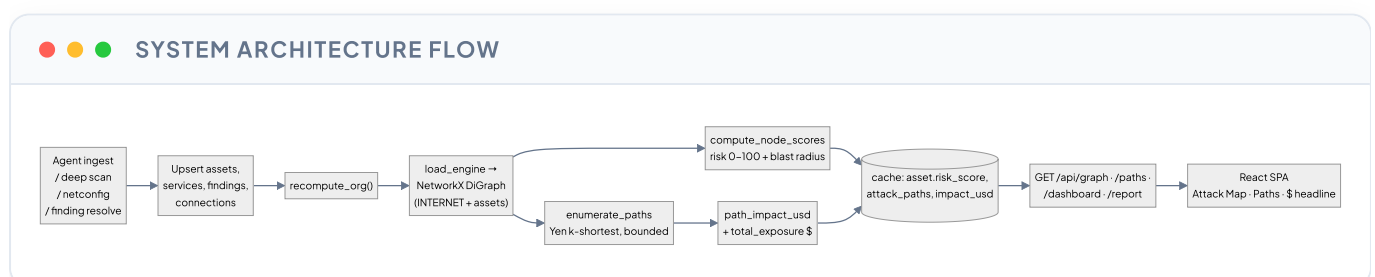


Router inventory (14 routers):

Router file	Prefix	Purpose
health.py	/	Liveness/readiness
auth.py	/api/auth	Register, login, refresh, profile
org.py	/api	Org info, members, load-sample, reset, agent-token
ingest.py	/api	Agent data ingestion
graph.py	/api	Attack map graph data
paths.py	/api	Attack path listing + detail
findings.py	/api	Finding CRUD + status transitions
assets.py	/api	Asset CRUD
ai.py	/api/ai	Remediation, impact, predict, block, network-summary
dashboard.py	/api	Dashboard + stats + recompute trigger
report.py	/api	CVE report, distribution, hardening, ML summary
live.py	/api	Live watch: devices, domains, threats, deep-scan, autoscan
netconfig.py	/api	NAT/DMZ/DHCP audit
urltrust.py	/api/url-analyzer	URL trust analysis

4. The core data flow: scan → price → visualize

The heart of the product. A change anywhere triggers a full recompute; the graph, paths, and dollars are always derived, never stored raw.



Why it feels alive: resolving a finding raises the edge weight on that hop, which lowers path likelihood, which lowers `impact_usd`, which lowers total exposure — all recomputed in one pass (`$902,900 → $702,900`, asserted by `make smoke`).

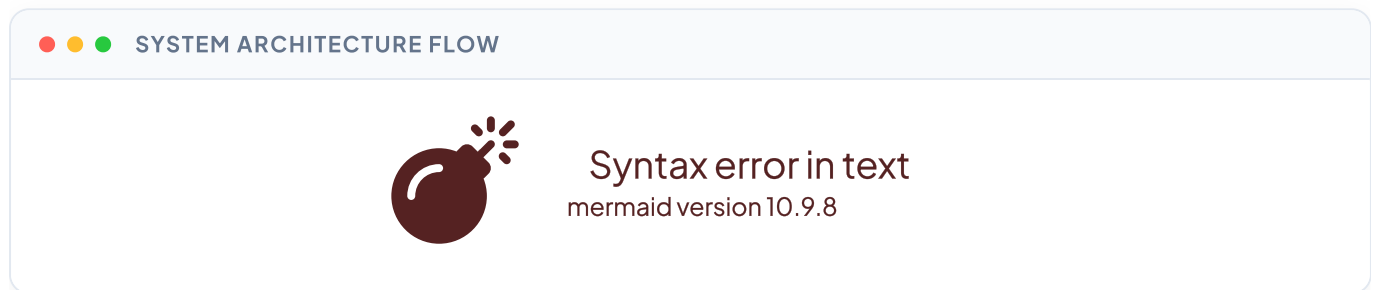
Recompute triggers (any of these calls `recompute_org`):

- Agent ingest (`POST /api/ingest`)
- Finding status change (`PATCH /api/findings/{id}`)
- Asset edit (`PATCH /api/assets/{id}`)
- Manual trigger (`POST /api/recompute`)
- Sample network load (`POST /api/org/load-sample`)

Concurrency control: Postgres advisory lock (`pg_advisory_xact_lock(hashtext(org_id))`) serializes concurrent recomputes for the same org. Skipped on SQLite.

5. Live network + threat overlay

How the *real* network reaches the Attack Map, and how threats light it up.

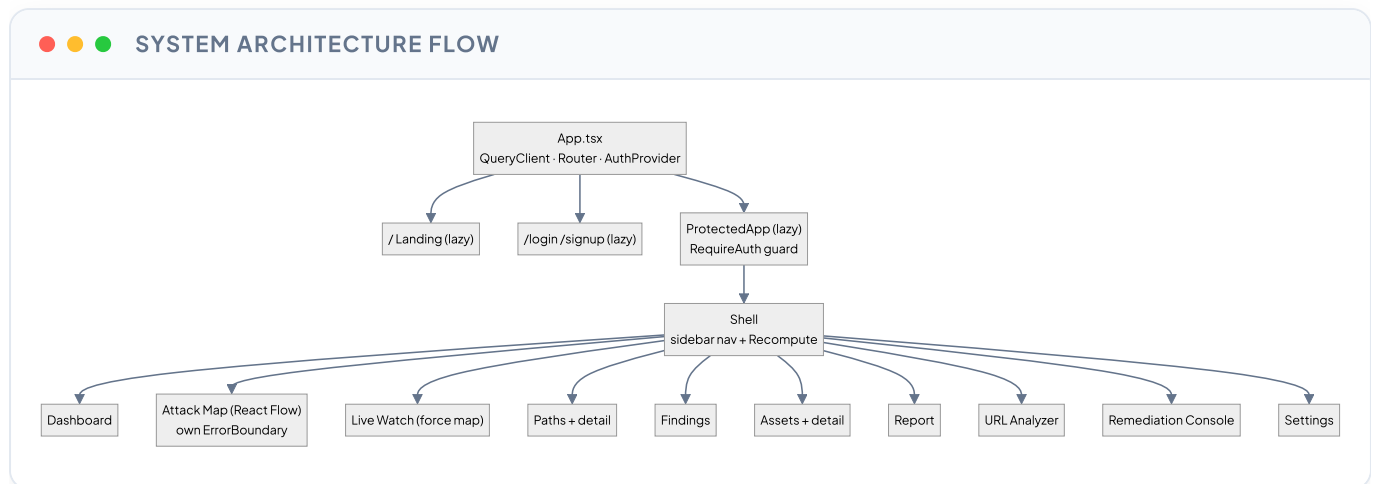


The gateway is **always** rendered (even after it's been deep-scanned into an asset); raw devices and deep-scanned assets hang off it. Threatened nodes carry `threat_kind` / `threat_severity` / `mitre` and pulse red on the map.

Live-only gate: once any `NetworkDevice` row exists for an org, the attack map filters to only show assets whose IP appears in the live device set (online + refreshed within 90s). This means:

- A fresh boot shows an empty map that fills only with real agent-discovered devices
- An offline machine drops off
- `DEMO_SEED=1` data renders without gating (no device rows exist yet)

6. Frontend structure



Code-split chunks (public vs app):

- Landing/auth are separate lazy chunks — the marketing page never downloads React Flow / Recharts
- `ProtectedApp` is lazy-loaded only after auth

State management:

- **TanStack Query** — server state (API calls, caching, refetching)
- **Zustand** — client state (toasts, graph focus state in `graphStore.ts`)

Key libraries:

- React Flow — attack map graph rendering
- Recharts — dashboard charts (severity distribution, etc.)
- Tailwind CSS — styling
- Motion (Framer Motion) — animations
- Vitest — testing

7. Deployment topology



**Local** (`make up`):

- Web: Vite dev server on `:5173`
- API: uvicorn on `:8000`
- DB: SQLite (`drishti.db`) or Postgres if `DATABASE_URL` set
- Agent: auto-started in `dev` mode (devices + WiFi discovery)

Production:

- Frontend: Vercel (static deploy from `web/`)
- Backend: Render (Docker from `server/Dockerfile` + `render.yaml`)
- Database: Managed Postgres (psycopg v3)
- CORS: explicit allowlist (no `*`)

Bootstrap on boot (`main.lifespan`):

1. `Base.metadata.create_all(engine)` — create missing tables
2. `reconcile_columns(engine)` — add missing columns additively
3. Backfill legacy device subnets (`backfill_device_subnets`)
4. Seed: identity-only (default) or Acme demo (`DEMO_SEED=1`)
5. Start autoscan scheduler (skipped under pytest)
6. Start Telegram alert dispatcher (if keys configured)
7. Auto-start edge agent (dev environments only)

Every step is try/except-wrapped so a failure logs and continues rather than blocking the app.

8. Key design decisions

Decision	Rationale	Source
Pure engine, separate persistence	<code>risk_engine</code> / <code>attack_paths</code> / <code>impact</code> are pure functions over <code>NetworkX</code> ; <code>recompute.py</code> owns all writes. Makes the math unit-testable and the demo explainable.	<code>risk_engine.py</code> , <code>recompute.py</code>
Engine-authoritative dollars	The AI narrates but the endpoint overwrites its number with the engine's — the figure is deterministic and auditable, never model-invented.	<code>impact.py</code> , <code>ai/service.py</code>
Live-only by default	Fresh boot seeds identity only; map/dashboard/live show <i>only</i> real agent-discovered devices. <code>DEMO_SEED=1</code> opts into the sample. No fabricated device ever ships.	<code>main.py</code> bootstrap, <code>read_service.py</code> live-only gate
Honesty model everywhere	netconfig <code>unknown</code> , deepscan <code>available:false</code> , urltrust <code>configured:false</code> — a check that can't run says so; it never fakes a pass or invents a CVE.	<code>netconfig/</code> , <code>deepscan/</code> , <code>urltrust/</code>
Defensive-only, consent-gated	No exploit path exists in code. LAN scans require consent + RFC1918 target. The agent discloses per-mode scope. AI output is guarded against offensive content.	<code>deepscan/service.py</code> , agent docstring, AI prompts
Bounded enumeration	Yen's k-shortest with hop/candidate/top-K caps — never enumerate the combinatorial simple-path set; recompute stays < 500 ms.	<code>attack_paths.py</code>
UUID-string PKs	36-char strings work identically on Postgres and SQLite, so local dev/tests and prod share one schema.	<code>models/base.py</code>
Additive <code>reconcile_columns</code> , no Alembic	Deliberate v1 simplification — adds missing columns safely so a stale dev DB doesn't 500; destructive changes recreate the DB.	<code>db_init.py</code>
Code-split frontend	Landing/auth/app are separate lazy chunks so the marketing page never pays for React Flow.	<code>App.tsx</code>
Nested error boundaries	A React Flow crash on the Attack Map is isolated; the rest of the app stays up.	<code>components/ErrorBoundary.tsx</code>
Timing-safe login	Unknown emails still run bcrypt against a precomputed dummy hash — response time	<code>core/security.py</code>

Decision	Rationale	Source
	doesn't leak account existence.	
Streaming body cap	<code>MaxBodySizeMiddleware</code> rejects >1MB bodies before buffering — protects <code>/ingest</code> from pre-auth DoS.	<code>main.py</code>
Deterministic graph layout	Server-side layered layout (zone-based columns, left→right) so the graph doesn't jump between renders.	<code>graph_layout.py</code>
Concurrent ingest safety	SAVEPOINT-based race handling — a concurrent upsert that wins the race is adopted instead of 500-ing.	<code>services/ingest.py</code>

9. Directory map (source of truth)

TEXT

```
server/app/  
  main.py app assembly, lifespan bootstrap, middleware  
  config.py env settings (fail-closed JWT secret)  
  db.py / db_init.py engine/session + create_all & reconcile_columns  
  core/ security (JWT, bcrypt, agent-token hash), deps (auth, rate limits), errors  
  api/ auth ingest graph paths findings ai dashboard report  
  live netconfig urltrust org assets health  
  services/  
    risk_engine.py graph, node scores, edge weights, blast radius ← pure  
    attack_paths.py Yen k-shortest enumeration, path risk/likelihood ← pure  
    impact.py $ model, total exposure ← pure  
    recompute.py orchestration: build → score → paths → impact → cache  
    engine_loader.py DB → engine NodeData/EdgeData  
    read_service.py build_graph (React Flow payload + live devices + threats)  
    ingest.py idempotent upsert + reconcile + recompute  
    ai/ client, prompts, service (remediate/impact/predict), mocks  
    deepscan/ scanner(nmap), cve_lookup(NVD/Vulners), parser, integration, service  
    netconfig/ facts, detectors(DMZ/NAT/DHCP), integration, service  
    urltrust/ analyzer, checks, network, providers, scoring, summary, whois  
    live.py device/domain observe, WiFi-aware tracking, coverage  
    live_threats.py detect_threats + demo inject/clear  
    autoscan.py per-org scheduled deep-scan  
    hardening.py per-node quantified hardening recommendations  
    dashboard_service.py dashboard + stats aggregation  
    accounts.py register, profile, org management, sample loading  
    telegram_alerts.py background Telegram notification dispatcher  
    intel.py (stub)  
  models/ 21 SQLAlchemy tables (see DATA_MODEL.md)  
  schemas/ Pydantic contracts (see API_REFERENCE.md)  
  seed/ acme.py (sample + identity), load.py  
web/src/  
  App.tsx / ProtectedApp.tsx routing + auth guard  
  features/* one folder per screen  
  components/ Shell, primitives, UI, ErrorBoundary  
  api/ client + types  
  store/ Zustand (toasts, graph)  
  lib/ format utilities  
agent/drishti_watch.py live watch edge agent (dns/history/conn/devices)
```

```
agent/drishti_agent.py legacy agent (minimal)
extension/ Chrome Web Guard (background, options, warning popup)
```

10. Background services

Service	File	Trigger	Frequency
Autoscan scheduler	services/autoscan.py	asyncio loop	Every 20s, checks due orgs
Telegram alerts	services/telegram_alerts.py	threading.Thread daemon	Every 30s, scans for new HIGH / CRITICAL findings + threats
Edge agent	agent/drishti_watch.py	Auto-started in dev	Configurable modes: dns, history, conn, devices

11. Configuration reference

Env var	Default	Purpose
APP_ENV	" " (treated as production)	local / dev / test allows default JWT secret
DATABASE_URL	sqlite:///./drishti.db	Database connection
JWT_SECRET	change-me	HS256 signing key (fail-closed in production)
JWT_ACCESS_MINUTES	15	Access token TTL
JWT_REFRESH_DAYS	7	Refresh token TTL
CORS_ORIGINS	http://localhost:5173	Comma-separated allowlist
AI_PROVIDER	nvidia	groq , nvidia , or anthropic
AI_MODEL	" " (provider default)	Override model name
AI MOCK	False	Use canned fixtures instead of real LLM
AI_MAX_TOKENS	2500	LLM response limit
AI_TIMEOUT_SECONDS	45.0	LLM call timeout
GROQ_API_KEY	" "	Groq API key
NVIDIA_API_KEY	" "	NVIDIA NIM API key
ANTHROPIC_API_KEY	" "	Anthropic API key
NVD_API_KEY	" "	NVD CVE lookup key (optional)
VULNERS_KEY	" "	Vulners CVE lookup key (alternative to NVD)
DEEPSCAN_TIMEOUT_SECONDS	120.0	nmap -sV timeout
DEEPSCAN_MAX_HOSTS	32	hosts per nmap batch
DEEPSCAN_MAX_TOTAL_HOSTS	256	hard ceiling across batches
BREACH_COST_BASE	500000.0	Dollar model base cost per breach
INGEST_MAX_BYTES	1048576 (1 MB)	Body size cap
AUTO_SEED	True	Seed org identity on fresh DB
DEMO_SEED	False	Seed Acme sample network on boot
TELEGRAM_BOT_TOKEN	" "	Telegram bot token (optional)

Env var	Default	Purpose
TELEGRAM_CHAT_ID	" "	Telegram chat ID (optional)